



Encapsulamento Visual — Como os Dados são Protegidos

Encapsulamento é a ideia de **esconder os dados internos** de uma classe e só permitir acesso controlado via métodos. Pense numa **caixa-fortinha**: você não pode mexer no que tá dentro diretamente, só usando a chave (os métodos).

SEM ENCAPSULAMENTO (ruim)

```
public class clientes {
    String nome; // ← público!
    qualquer um acessa
}
```

Risco: outro código pode fazer `cli.nome = null;` e destruir o dado sem você poder impedir.



COM ENCAPSULAMENTO (bom)

```
public class clientes {
    private String nome; // ← só visível dentro da classe
    public void setNome(String n) {
        if (n != null && n.length() > 2)
        {
            this.nome = n;
        }
    }
    public String getNome() { return nome; }
}
```

Controle: você PODE validar no setter (ex: nome deve ter mais de 2 letras). Ninguém acessa o campo diretamente.

O que o encapsulamento permite fazer

Operação	Como é feito	Exemplo neste projeto
Ler dado	Via método <code>get</code>	<code>cli.getNome()</code> → retorna o nome
Escrever dado	Via método <code>set</code>	<code>cli.setNome("Maria")</code> → guarda o nome
Validar antes de guardar	Inside o <code>set</code> , checa condições	não implementado ainda (exercício!)
Mudar implementação internamente	Só muda dentro da classe, o resto do programa não quebra	se amanhã CPF virar <code>int</code> , só muda <code>Cpf.java</code>

Diagrama visual: camadas de acesso

CÓDIGO EXTERNO

outras classes, main, etc.

```
cli.setNome("Maria") ← chama método público  
String n = cli.getNome() ← chama método público
```

CLASSE clientes

(métodos públicos: setNome, getNome)

```
public void setNome(String n) {  
    this.nome = n; // acessa campo privado  
}
```

```
public String getNome() {  
    return nome; // acessa campo privado  
}
```

DADOS PRIVADOS

```
private String nome; ← ninguém de fora acessa direto!
```



barreira de proteção



💡 **Dica:** no projeto atual os campos são package-private (sem modificador — nem private nem public). Funciona para a aula, mas o padrão correto é **private** com getters/setters públicos. Quando você avançar, mude todos os campos para **private**.

Por que o **this**. é importante?

```
public void setNome(String nome) { // parâmetro se chama "nome"  
    this.nome = nome; // this.nome = campo da classe  
}
```

Sem o **this**:

```
nome = nome; ← erro! atribui o parâmetro a ele mesmo (sem sentido)
```

O **this** é uma referência para "este objeto" — o próprio objeto que tá na memória.

Exercício 6 — Tornar campos privados (encapsulamento completo)

1. Abra cada classe Java: clientes.java, endereços.java, Telefone.java, Cpf.java, Cnpj.java
2. Mude todos os campos de String x; para private String x;

3. Compile e rode — deve continuar funcionando porque os getters/setters são públicos
4. Tente acessar direto: `cli.nome = "X"`; no main — o NetBeans deve mostrar erro de compilacao
5. Agora usa apenas `cli.setNome("X")`; — funciona
6. Salve e comente: "agora os dados estão protegidos"

Exercício 7 — Validação no setter (nome com pelo menos 3 letras)

1. Abra `clientes.java`
2. Altere o `setNome` para:

```
public void setNome(String nome) { if (nome != null && nome.trim().length() >= 3) { this.nome = nome; }  
else { System.out.println("ERRO: nome deve ter pelo menos 3 caracteres!"); } }
```
3. No `0o02.java`, tente cadastrar com nome de 2 letras (ex: "Ab")
4. O programa deve imprimir o erro mas continuar rodando
5. Teste com nome de 3+ letras (ex: "Ana") — deve funcionar
6. Adicione `System.out.println("Nome cadastrado: " + cli.getNome());` depois do `setNome` para confirmar

Exercício 8 — Validar CEP com 8 dígitos

1. Abra `endereços.java`
2. Altere o `setCep` para:

```
public void setCep(String cep) { String limpo = cep.replaceAll("[^0-9]", ""); if (limpo.length() == 8) {  
this.Cep = limpo; } else { System.out.println("ERRO: CEP deve ter 8 dígitos!"); } }
```
3. No `0o02.java`, tente cadastrar CEP com 7 ou 9 dígitos
4. Tente com CEP válido de 8 dígitos (ex: "01000000")
5. O programa deve aceitar só o correto

Exercício 9 — Adicionar data de cadastro automática

1. Crie um campo `String dataCadastro`; na classe `clientes`
2. No `setNome`, ao confirmar o nome, também preencha a data com `new java.text.SimpleDateFormat("dd/MM/yyyy").format(new java.util.Date())`
3. Ou use `java.time.LocalDate.now()` (mais moderno)
4. Adicione `getDataCadastro()` para ler a data
5. No `0o02.java`, imprima a data de cadastro junto com os outros dados
6. Resultado: quando você cadastra um cliente, o sistema guarda automaticamente a data de hoje

Exercício 10 — Cliente com telefone MULTIPLO

1. Crie uma classe `ListaTelefones` que guarda vários telefones (use `ArrayList<Telefone>`)
2. Ou, de forma mais simples: adicione campos `String tel1`; `String tel2`; na classe `clientes`
3. No `0o02.java`, pergunte 2 telefones ao usuário
4. Na listagem de CPF, imprima os 2 telefones
5. **Desafio avançado:** use `java.util.ArrayList<Telefone>` para ter uma lista infinita de telefones